Compte rendu : TP 2

Réalisé par LOARI Moumouni Boukary

1. Architecture réalisée

Une **AC racine** (P-384, SHA-384, 10 ans, hors ligne, pathlen:1), une **AC intermédiaire** (P-256, 5 ans, pathlen:0, limitée à tp.local par nameConstraints) et des certificats finaux P-256 de 90 jours : serveur www.tp.local (SAN DNS:www.tp.local, DNS:tp.local) et clients alice@tp.local et bob@tp.local (SAN email:). racine.cnf et inter.cnf sont ceux de l'annexe B ; pki.sh construit l'ensemble (set -euo pipefail).

Deux ajouts signalés dans les fichiers : unique_subject = no (sans lui openssl ca refuse un second certificat pour le même nom, et le renouvellement du TP 3 échoue) et la section ext_client_csr (SAN repris de la demande, pour Bob).

2. Rôle des lignes d'extensions (étape 1)

Ligne	Rôle
basicConstraints = critical,CA:TRUE,pathlen:0	C'est une AC, mais elle ne signe que des certificats finaux, jamais une autre AC
keyUsage = critical,keyCertSign,cRLSign	Sa clé ne sert qu'à signer certificats et CRL
nameConstraints = critical,permitted;DNS:.tp.local,DNS:tp.local,email:tp.local	Valable uniquement pour tp.local, ses sous-domaines et ses courriels ; critical : refus par un client qui ne comprend pas l'extension
subjectKeyIdentifier / authorityKeyIdentifier	Identifiants de clés servant à construire la chaîne
ext_serveur / ext_client : CA:FALSE, digitalSignature, extendedKeyUsage = serverAuth / clientAuth	Certificats finaux limités à l'usage serveur TLS ou client TLS
copy_extensions = copy (intermédiaire) / none (racine)	L'intermédiaire reprend les SAN de la CSR ; la racine décide seule des extensions

Ligne	Rôle
default_md, default_crl_days, policy	SHA-384 / 30 j (racine), SHA-256 / 7 j (intermédiaire) ; seul le commonName est obligatoire

3. Validations (étapes 2 à 5)

Commande	Résultat	Interprétation
verify -verify_hostname www.tp.local serveur.crt	OK	Chaîne valide, nom dans le SAN
idem avec autre.example	error 62 hostname mismatch	Chaîne valide, nom différent
verify -purpose sslclient alice.crt	OK	Usage client autorisé
verify -purpose sslserver alice.crt	error 26 unsuitable purpose	Certificat clientAuth refusé comme serveur
verify d'un certificat www.example.com	error 47 permitted subtree violation	Nom hors de tp.local

openssl ca a pourtant signé le certificat www.example.com : signer est un acte mécanique, ce sont les **validateurs** qui appliquent les contraintes de l'intermédiaire. Même compromise, elle ne peut donc pas faire accepter un nom hors tp.local.

4. Révocation (étape 6) et automatisation (étape 7)

openssl ca -revoke alice.crt -crl_reason keyCompromise marque la ligne d'index.txt à l'état R (série 1001). Les CRL de l'intermédiaire (7 jours) et de la racine sont concaténées dans crls.pem. Avec -crl_check_all, alice.crt donne error 23 certificate revoked. La CRL de l'intermédiaire indique Last Update et Next Update espacés de 7 jours, le numéro de CRL 4096 (1000 en hexadécimal), la série 1001 et la raison Key Compromise. serveur.crt, lui, reste accepté (OK). OpenSSL exige une CRL **pour chaque niveau** de la chaîne : avec la seule CRL de l'intermédiaire, même un certificat valide est refusé (unable to get certificate CRL).

5. Questions de réflexion

1. **Racine hors ligne ; compromission ?** Tout repose sur sa clé : support déconnecté, usage rare. Intermédiaire compromise : on la révoque depuis la racine, on en crée une nouvelle et on réémet

les feuilles. Racine compromise : pas de remède local, il faut une nouvelle racine redistribuée dans tous les magasins de confiance.

2. **pathlen:0 ?** Il interdit à l'intermédiaire de créer une AC enfant. Sans lui, qui détient sa clé pourrait fabriquer une AC hors politique et la chaîne resterait valide.
3. **Clé générée par le demandeur ?** Une clé privée ne doit exister qu'à un endroit. La CSR, signée avec la clé, prouve la possession sans que la clé circule ; si l'AC la générait, elle la connaîtrait et il faudrait la transmettre.
4. **CN ou SAN ?** Le **SAN** : la RFC 6125 a rendu le CN obsolète, et OpenSSL comme les navigateurs l'ignorent dès qu'un SAN existe.
5. **Limites des CRL ?** Elles grossissent, sont publiées périodiquement (un certificat révoqué reste accepté jusqu'à la mise à jour) et un client qui ne peut les obtenir les ignore souvent. OCSP donne l'état d'un certificat en quasi temps réel mais coûte une requête par connexion (agrafage possible). Des certificats de très courte durée rendent la révocation presque inutile, au prix d'un renouvellement automatisé (TP 3).
6. **Politique minimale.** Les certificats peuvent être demandés par les administrateurs du domaine `tp.local` et par les utilisateurs inscrits de l'université, par l'intermédiaire de leur responsable. L'identité est vérifiée selon le cas : pour un serveur, par la preuve du contrôle du nom (accès à la machine ou DNS), et pour un client, par le service informatique sur pièce justificative. Les noms délivrés sont uniquement ceux placés sous `tp.local`, ce que garantit la contrainte de nom de l'intermédiaire. Les clés sont générées par le demandeur, en P-256 au minimum, et ne sont jamais transmises à l'AC. Les certificats finaux sont valables 90 jours, l'intermédiaire 5 ans et la racine 10 ans. Un certificat sert à un seul usage, serveur ou client, et aucune feuille ne peut être une AC. La révocation est faite sur demande du titulaire, dès suspicion de compromission ou au départ de l'utilisateur, et traitée sous 24 heures. La CRL de l'intermédiaire est mise à jour au moins tous les 7 jours, et après chaque révocation. La clé de la racine reste hors ligne et sauvegardée, celle de l'intermédiaire est conservée sur un poste restreint ; toute émission et toute révocation sont consignées (`index.txt`).